

Runtime-Governed Plan–Propose–Verify Agents: A Harness for Reliable Behavior Under Uncertainty

Team FAIR (Naman Goel, Jordan Langham-Lopez, Kausik Lakkaraju, Lukasz Szpruch)

The Alan Turing Institute, London, UK

Contact email — ngoel@turing.ac.uk

Abstract

CAR-bench tests tool-using agents under ordinary task demands, manipulated capabilities, and ambiguous user requests. We present a runtime-governed plan–execute–verify harness. Separately prompted planner, proposer, and verifier roles exchange a structured plan, evidence state, live-schema diagnostics, and bounded repair decisions. Deterministic controls parse their outputs, validate proposed calls, contain literal unknown results, preload available preferences, enforce a five-call depth bound, and produce grounded fallbacks when output cannot safely be used. Most mechanisms in the harness derive their working state from the live A2A interface, making the harness portable beyond CAR-bench.

1 Introduction

Reliable in-car tool use requires more than schema-valid JSON. An agent must infer intent, retrieve missing state, obey policy, avoid unsupported effects, and communicate uncertainty. CAR-bench exposes these demands through base, hallucination, and disambiguation tasks over a mutable automotive environment [Kirmayr et al., 2026]. Failures can arise from an invented argument, a valid but insufficient call, unnecessary clarification, or several individually valid route edits executed against stale state. Similar issues arise beyond CAR-bench as well [Schick et al., 2023, Yao et al., 2025].

Our design separates semantic judgment from mechanically checkable invariants, but governance is present throughout the sequence, not only at final output. Role-specific prompts expose policy obligations, evidence, completion criteria, confirmation requirements, and forbidden assumptions. Runtime code maintains plan and evidence state, checks live tool surfaces, routes bounded repair, prevents unresolved observations from becoming asserted facts, and records model calls and local interventions. The model remains responsible for intent, policy applicability, and action sufficiency.

2 Method: Layered Runtime Governance

2.1 Plan–Execute–Verify Core

For each A2A agent step, the planner-role call is supplied with the conversation, structured context, public policy, and

live tool schemas. Its structured suggested plan records candidate tools and argument surfaces, evidence needed, preconditions, completion criteria, confirmation requirements, clarification conditions, dependencies, and forbidden assumptions. The proposer-role call receives that plan plus updated evidence and is prompted to emit exactly one next action: tool calls, a clarification question, or a terminal response. The verifier-role call receives the same grounding and is prompted to assess sufficiency, policy compliance, ordering, confirmation specificity, unnecessary clarification, over-refusal, and unsupported claims. Runtime code parses its acceptance or single defect and opens only the corresponding bounded repair [Yao et al., 2023; Shinn et al., 2023].

2.2 Evidence, Plan, and Schema Governance

Before each role call, the harness compacts public A2A information into structured context: live schemas, recent outcomes, reusable facts, result paths, errors, literal unknown fields, and weak plan-progress indicators. When the live preference-read tool is available, one schema-valid read-only preload makes its result visible in the ordinary transcript. Plan tool surfaces are then annotated against the current schema so unsupported names or arguments become explicit diagnostics rather than executable suggestions.

Every proposed call is checked for an exact live tool name, required and undeclared arguments, JSON types, enums, and nested structure. A schema-invalid action may receive one bounded correction and is revalidated before exposure; a still-invalid repair becomes a grounded user-facing limitation, never a tool call. Robust JSON recovery is used only to recover a complete role schema, and the official action parser remains authoritative. Internal planning-helper calls are removed from visible output. These checks are category-blind and use no gold catalogue or manipulated-tool detector. Schema validity remains necessary, not sufficient: semantic capability coverage stays with the prompted roles.

2.3 Literal Unknown-Result Containment

Tool result fields that literally contain “unknown” are retained as unresolved evidence. The harness blocks those values from being copied into later tool arguments, requires user-facing acknowledgement when relevant, and prevents claims contradicted by the unresolved observation. A later known observation or concrete successful result can retire the

record. Repeating the same zero-argument lookup is stopped only after the same result path has returned unknown twice consecutively. This mechanism does not infer arbitrary missing capability or assume that one unknown field blocks an otherwise independent action.

2.4 Additional Deterministic Controls Examples

The final-output guard activates when the live system policy explicitly requires 24-hour datetime text. It converts AM/PM expressions in assistant responses and user-visible email content, while leaving calendar/tool arguments and already-24-hour text unchanged. There were a few other such deterministic guards we considered but decided to keep them out from the submission due to various reasons. See Section 4.1.

2.5 Policy, Confirmation, and Disambiguation Governance

The role prompts are the main policy and disambiguation layer. The planner prompt instructs the model to turn each applicable live-policy obligation into evidence, ordering, confirmation, or completion items. It must first check the conversation, structured context, preferences, current results, and an available read-only lookup before asking for an objective fact. The proposer prompt treats the plan as a checklist but is instructed that newer transcript evidence and live schemas override it. The verifier prompt is instructed to reject unsupported guessing, omitted obligations, wrong evidence sources, premature success, and unnecessary clarification, while retaining a question when a valid subjective choice genuinely belongs to the user.

This asymmetry matters: retrieving an objective fact is preferred when it cannot change the world, whereas a state-changing tool is never used merely to test an interpretation. When confirmation is required, the prompts require the concrete action, target, and key value or route, and prohibit reusing confirmation for a materially different call. Prompt governance guides model judgment; deterministic parsing and call validation ensure that malformed or undeclared actions cannot bypass it.

3 Compute-Time-Aware Inference Scaling

3.1 Conditional Compute Allocation

The architecture exploits fast inference through specialization rather than unconditional parallelism. The normal path invokes planner-, proposer-, and verifier-role calls. Additional inference is conditional on the parsed verifier-role response: one proposer-repair call for a local action error, or a planner-repair call plus a proposer rerun for a strategic defect [Madaan et al., 2023]. Repaired output is not reviewed again. Schema repair also consumes a numbered slot and cannot form a nested loop. Evidence assembly, schema checks, unknown-result handling, planning-helper removal, and time normalization run locally and consume no LLM-call slot. All experiments were conducted using Cerebras hosted gpt-oss120b model (for all agent models as well as evaluator/user model). We have tried to exploit inference-time scaling

whenever useful, rather than minimizing it. Caching was taken into account while designing prompts.

Planner- and verifier-role calls use high reasoning with 8,192-token completion limits because their prompts ask the model to integrate policy, state, tools, and goal coverage. Proposer-role calls use medium reasoning and 3,072 tokens because their prompt requests a compact action. All roles use temperature 0.1. Unconditional parallel planner calls and expert-verifier ensembles were not retained: they raised compute and latency without a reliable aggregate accuracy gain.

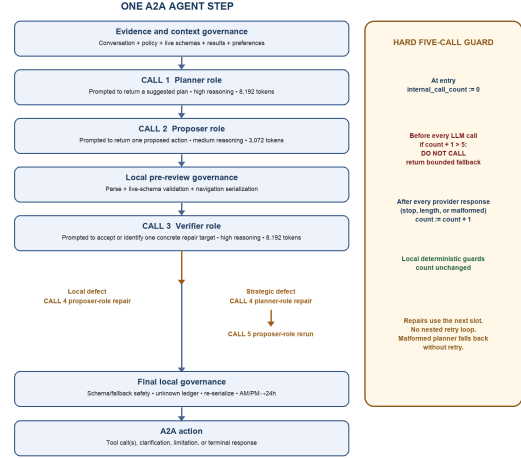


Figure 1: Per-step architecture, local guards, and maximum-depth.

3.2 Hard Depth Bound and Audit

A shared internal counter permits at most five sequential LLM calls per A2A step. It starts at zero, is checked before every provider call, and increments after every attempted model response, including malformed or length-limited output. The maximum strategic path is planner call → proposer call → verifier call → planner-repair call → proposer-rerun call. Malformed planner-role output is not retried: the harness constructs bounded fallback guidance for the proposer prompt. Malformed proposer output may use one available retry. If verifier-directed repair is unavailable, malformed, unchanged, or still schema-invalid, the runtime produces a grounded terminal fallback rather than executing it or opening another loop.

Track 2 usage is reported through the existing A2A turn-metrics fields for prompt, completion, and thinking tokens. Across full multi-turn tasks the observed mean is 13.03 internal LLM calls, while no individual A2A step exceeds five sequential calls. Mean total usage is 458,858 tokens per task, below the 500K average-task limit. Provider wait and evaluator time are excluded from reported model latency.

3.2 Track 1 Variation

For track 1, we have used the same architecture, with only the following changes. We replaced gpt-oss120b with GPT-5.6 Sol. The maximum depth (as discussed in 3.2) was increased to 6. Output token limits were also increased for planner-, proposer- and verifier- roles.

4 Development Evaluation

We ran three independent repeats on all public train and validation tasks with identical model and semantic-core settings. Table 1 reports task-weighted mean accuracy and Pass³, where a task passes only if all three repeats succeed [Yao et al., 2025]. Across three repeats (all else equal), task-weighted mean accuracy is 63.6% on train and 62.1% on validation; Pass³ is 44.2% and 44.0%. Mean Track 2 usage was 458,858 tokens (with significant fraction cached) and 13.03 LLM calls per task. For the disambiguation category, we observe a shift on validation set performance, which could be due to small sample size of the validation set.

Category	Train mean	Train Pass ³	Val. mean	Val. Pass ³
Base	75.3	58.0	70.7	54.0
Hallucination	57.6	39.6	52.7	32.0
Disambiguation	53.8	29.0	64.0	48.0
Overall	63.6	44.2	62.1	44.0

Table 1: Three-repeat public-set accuracy (%) and Pass³ consistency.

Input	Cached input	Reasoning	Visible output	Total	Calls/task
412,261	197,841	22,440	24,158	458,858	13.03

Table 2: Mean Track 2 model compute and latency per completed task.

4.1 Ablations and Lessons

Development ablations varied prompt density, planner/reviewer parallelism, capability-coverage prompting, repair behavior, and deterministic fallbacks. In our experiments, parallel planner calls increased tokens without a reliable accuracy gain, while aggressive prompt compaction could remove useful, repeated constraints. Schema validation reduced invented names and malformed arguments but could not determine whether a valid call was sufficient for the user’s goal. In another deterministic intervention, when a proposer-role response proposes several navigation mutations, the runtime could forward only the first edit. Non-navigation calls would remain unchanged, and the suggested plan retained in runtime state so a later model call could reconsider the deferred edit after the first tool result updates route state. Serialization would be applied before the verifier call and after accepted repairs, preventing a repair from reintroducing a stale-state batch. It did not synthesize route identifiers, select destination, or block a navigation edit. Taking this to an extreme, we testes another version that attempted to write entire policy as code. The code was not derived from the evaluator logic; instead, it was constructed from the natural language policy available. This version of the harness seemed to significantly improve performance on base tasks but regress on other tasks, overall leading to no net gain or loss compared to the submitted version. The final design therefore retains the broad semantic core governance—structured planning, evidence propagation, live-schema checks, review, bounded repair, unknown containment, fallbacks, and audit.

Our observations do not argue against more deterministic checks and guards. They show that a check is only as reliable as the information available to define its trigger, scope, and consequence. For example, if fan-speed control is unavailable, a whole-request blocker may also abandon independent requests to close windows or change air circulation. Conversely, allowing every supported sub-action can be wrong when all actions jointly implement one indivisible requested effect. The retained deterministic guard has exact local semantics: normalize explicit AM/PM text under an applicable format policy.

We have not reported detailed results for different baselines and ablations because ablations and comparison are mostly based on small test runs (due to compute and API budget constraints) and there was instability even in full runs pass³. We were also not able to conduct evaluations with models other than gpt-oss120b, due to the budget constraint. Given more budget, we would study these questions in further detail.

4.2 Diagnostics

Each run preserves trajectories, raw provider request/response records, tool observations, finish reasons, transport errors, and per-turn tokens. Diagnostics distinguish malformed model output, schema repair, unsupported plan surfaces, preference preload, unknown-result containment, fallback paths, and rejected unchanged repairs.

In addition to accuracy metrics, these traces from the training set were used for qualitative assessment while designing the harness, to decide which changes to keep and which ones to discard.

4.3 Category-Level Design Tensions

The categories reward different error preferences. Base tasks favor decisive multi-step completion; excessive confirmation, repeated lookup, or conservative termination can reduce success. Hallucination tasks reward calibrated refusal when an essential capability is absent, yet the live schema may not expose what was removed. Disambiguation tasks require retrieval of objective facts while avoiding invention of subjective preferences. A global rule such as “always ask when uncertain” only moves errors between categories.

The harness uses a hybrid boundary: prompts assess semantic sufficiency; code enforces only fully grounded invariants. Orchestration, evidence state, schema checks, repair, budgets, and fallbacks are populated chiefly from live A2A policy, schemas, transcript, and results—not a stored CAR-bench answer catalogue. Prompts and two local controls remain domain-specific, so this is useful portability rather than universal benchmark independence. No rule uses task labels, evaluator output, or an expected-action lookup.

5 Error Analysis and Limitations

5.1 Structural Validity Is Not Sufficiency

The largest remaining weakness is hallucination limit-awareness. The runtime validator is given only the live schema. Conformance to that schema is not proof that the requested

effect is achievable. A deterministic layer cannot construct a faithful missing-capability explanation when neither policy nor public runtime metadata states what is absent. Broad blocking can protect one task while unnecessarily abandoning valid work in another.

This boundary is clearest when an argument or tool is removed but the remaining interface stays coherent. The validator can establish that an undeclared argument is invalid; the schema alone cannot establish that the omitted effect was essential. Comparing against a hard-coded gold catalogue would amount to detecting manipulation rather than solving the user's task. The role prompts must therefore elicit goal-to-capability reasoning from the model. Reliability could improve through stronger models, or through richer public metadata that makes the relationship between requested effects and tool capabilities explicit.

5.2 Why Deterministic Policy Enforcement Is Challenging

Deterministic policy enforcement is attractive because an explicit obligation can be checked consistently and without another model call. Pass³ metric also rewards determinism. However, the submitted version does not encode the complete policy in code: prompts interpret it, while code enforces only unambiguous triggers and consequences. A sunroof rule still needs its trigger, sunshade state, percentage, ordering, and weather-dependent confirmation condition. A defrost rule needs the joint or partial semantics of its airflow, air-conditioning, fan-speed, and window obligations. Navigation state can change after the first mutation, invalidating precomputed later edits. As discussed in 4.1, our attempts to encode these details did not yield reliable improvements within the competition timeline, but it is something worth exploring in future work. Policy code needs precise contracts: structured triggers, prerequisites, exceptions, ordering, postconditions, scope, and partial-completion semantics; tools need effects, mutability, confirmation, dependencies, and returned post-state. A deterministic reasoner could then maintain an evidence/obligation graph, derive preconditions, and surface them to role prompts, leaving ambiguous applicability and intent to the model. Until such metadata exists, grounding the general harness in live A2A information limits hidden assumptions; the two local controls in the submitted version are deliberately small examples. While these two local controls didn't contribute to improving the performance in our experiments, they also didn't hurt and therefore, were kept in the submission. At the same time, a potential advantage of such low-code harnesses is that they may be more portable across models and benchmarks.

5.3 Operational Failure Modes

Length-limited high-reasoning responses remain an operational risk. Larger completion limits improve recovery but can threaten the average-token constraint. The five-call counter prevents retries and repairs from forming nested loops. When planner-role output is malformed, the harness immediately constructs bounded fallback guidance; it does not issue

another planner call. Malformed proposer-role output may be retried only while a numbered slot remains. Partial model text is used only if it parses into the complete schema required for that role. An incomplete proposer-role response is never executed as an action. Rate-limit and transport failures use bounded backoff outside semantic repair, while raw interactions and unresolved requests remain auditable. A transport timeout bounds provider waiting but does not change evaluator logic or the A2A contract.

6 Reproducibility and Compliance

The submission is packaged in a public digest-pinned container. Model name, API base/key, temperature, reasoning efforts, completion limits, timeout, and repair profile are supplied through environment variables. The official evaluator remains external and unmodified. The agent reports tokens only through existing A2A metrics and never reads evaluator code, task labels, task-level scores or other hidden logic. The reported development runs used Cerebras-hosted gpt-oss120b (including user/evaluator simulator), under organizer-provided rate limits. Network variability affects wall-clock latency but not semantic call depth. While overall pass³ is stable, there is variance in other numbers reported in Table 1, due to multiple sources of model-induced randomness in the agent as well as in the evaluator.

A task is never re-prompted against an official score. The two deterministic rules inspect only the proposed action, conversation, live policy text, and ordinary public tool names required to recognize navigation edits or visible email content.

7 Conclusion

CAR-bench shows that reliable tool agents need both model intelligence and disciplined runtime control. Our submission implements layered governance across the complete plan-execute-verify sequence: structured obligations and completion criteria, compact evidence state, live plan-surface annotation, action-schema validation, bounded review and repair, preference grounding, and terminal fallbacks. These general mechanisms are driven primarily by live A2A information. The harness is low-code, uses live information and is thus, portable. It reserves model judgment for intent, policy applicability, and capability sufficiency.

For future work, in addition to hardening the harness code, it would be useful to also think how tool meta-data and policy can be made available in a better (ideally, machine readable) format, which in turn can be used for reliable runtime deterministic reasoning and for *auto-harness* [Lou et al, 2026; Ning et al 2026]. It would also require clarifying the assumptions needed for deterministic runtime governance to work (for e.g. assumption about availability of clear policy and tool descriptions, that can then be used to write policy as code). Better exploiting training data for harness design (or auto-harness) is another interesting avenue for future exploration. Beyond Pass³, considering better metrics for cost (e.g. token usage) as well as for meaningful human interaction is also an important direction to explore.

References

- [Kirmayr et al., 2026] Johannes Kirmayr, Lukas Stappen, and Elisabeth André. CAR-bench: Evaluating the Consistency and Limit-Awareness of LLM Agents under Real-World Uncertainty. In Proceedings of the ACL, 2026.
- [Madaan et al., 2023] Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegrefe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, Shashank Gupta, Bodhisattwa Prasad Majumder, Katherine Hermann, Sean Welleck, Amir Yazdanbakhsh, and Peter Clark. Self-Refine: Iterative Refinement with Self-Feedback. In Advances in Neural Information Processing Systems (NeurIPS), 2023.
- [Schick et al., 2023] Timo Schick, Jane Dwivedi-Yu, Roberto Dessi, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language Models Can Teach Themselves to Use Tools. In Advances in Neural Information Processing Systems (NeurIPS), 2023.
- [Shinn et al., 2023] Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language Agents with Verbal Reinforcement Learning. In Advances in Neural Information Processing Systems (NeurIPS), 2023.
- [Yao et al., 2023] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing Reasoning and Acting in Language Models. In Proceedings of ICLR, 2023.
- [Yao et al., 2025] Shunyu Yao, Noah Shinn, Pedram Razavi, and Karthik Narasimhan. τ -bench: A Benchmark for Tool-Agent-User Interaction in Real-World Domains. In Proceedings of ICLR, 2025.
- [Lou et al, 2026] Lou, Xinghua, et al. "Autoharness: improving llm agents by automatically synthesizing a code harness." arXiv preprint arXiv:2603.03329 (2026).
- [Ning et al, 2026] Ning, X., Tieu, K., Fu, D., Wei, T., Li, Z., Bei, Y., ... & He, J. (2026). Code as agent harness. arXiv preprint arXiv:2605.18747.